

Vaja 2 – Uporaba genetskega algoritma za optimizacijo izraza

Ime in priimek: **Gašper Leskovec**

Predmet: Inteligentni sistemi za podporo odločanju

Datum: 27. julij 2026

1 Uvod

Cilj naloge je poiskati aritmetični izraz, sestavljen iz števk 1–9 in osnovnih operacij (+, −, *, /), katerega vrednost ustreza izbrani ciljni vrednosti. Iskanje je izvedeno z genetskim algoritmom (GA), brez uporabe vgrajenih Matlabovih orodij za optimizacijo.

2 Opis naloge in metode

Rešitev je predstavljena kot zaporedje izmenjujočih se števk in operatorjev (npr. $5+7+9+4*8$) dolžine, ki jo določa parameter `max_terms`. Uporabnik ob zagonu poda ciljno vrednost in največje število členov izraza; velikost populacije, največje število generacij in verjetnost mutacije so nastavljene kot konstante na vrhu skripte.

Algoritem sledi standardni zgradbi genetskega algoritma:

- **Inicializacija:** začetna populacija je sestavljena iz naključno generiranih izrazov ustrezne dolžine.
- **Ocenjevanje:** vsak izraz se ovrednoti (z varno obravnavo neveljavnih izrazov, npr. deljenja z 0); kriterijska funkcija je definirana kot $1/(1 + |cilj - vrednost|)$, kar da vrednost 1 pri natančnem zadetku in gladko pada proti 0 z oddaljenostjo od cilja.
- **Selekcija:** starši za naslednjo generacijo se izbirajo z metodo ruletnega kolesa, kjer je verjetnost izbire osebkov sorazmerna z njegovo kriterijsko funkcijo.
- **Križanje:** dva starša tvorita dva otroka z izmenjavo delov izraza na naključni točki.
- **Mutacija:** vsak gen (števka ali operator) se z verjetnostjo `mutation_prob` naključno spremeni, kar vzdržuje diverziteto populacije.

- **Elitizem:** najboljši osebek generacije se nespremenjen prenese naprej, zato najboljša dosežena kriterijska funkcija nikoli ne pade.

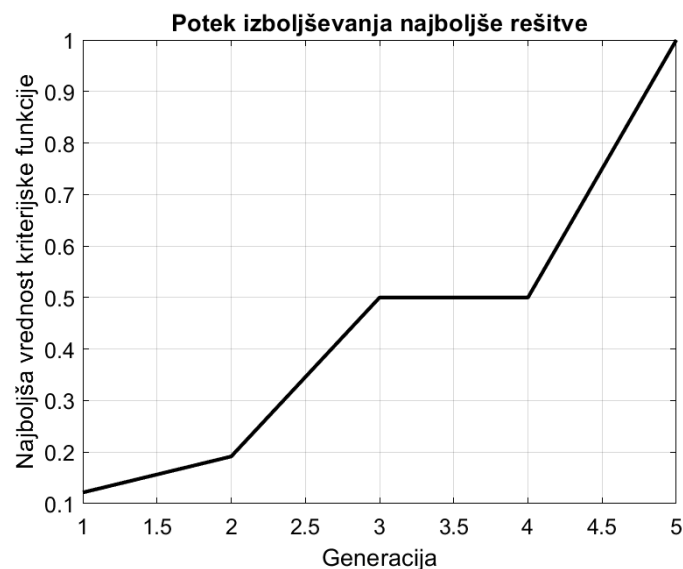
Postopek se ponavlja do največjega dovoljenega števila generacij ali dokler algoritem ne najde izraza, ki natančno doseže ciljno vrednost.

3 Rezultati glavnega zagona

Glavni zagon je izveden s parametri: ciljna vrednost 53, `max_terms` 5, populacija 20, `max_generations` 100, `mutation_prob` 0.1.

Generacija	Izraz	Vrednost	Kriterijska funkcija
1	$3*5*3+7/9$	45.7778	0.1216
2	$3+5*9+7/9$	48.7778	0.1915
3	$6+7+9+4*8$	54.0000	0.5000
4	$6+7+9+4*8$	54.0000	0.5000
5	$5+7+9+4*8$	53.0000	1.0000

Tabela 1: Najboljša rešitev po generacijah (glavni zagon)



Slika 1: Potek izboljševanja najboljše rešitve skozi generacije

Algoritem je ciljno vrednost dosegel v peti generaciji. V prvih dveh generacijah je bil najboljši osebek še razmeroma daleč od cilja ($3*5*3+7/9 = 45.7778$ s kriterijsko funkcijo 0.1216 oziroma $3+5*9+7/9 = 48.7778$ s kriterijsko funkcijo 0.1915), v tretji generaciji se je približal na $6+7+9+4*8 = 54$ (kriterijska funkcija 0.5000), v peti generaciji pa je mutacija vodilne številke

iz 6 v 5 dala natančen zadetek $5+7+9+4*8 = 53$ s kriterijsko funkcijo 1.0. Potek kriterijske funkcije je skozi vse generacije nepadajoč ($0.1216 \rightarrow 0.1915 \rightarrow 0.5000 \rightarrow 0.5000 \rightarrow 1.0000$), kar je neposredna posledica elitizma: v četrti generaciji ni bilo izboljšave, a najboljša rešitev se tudi ni poslabšala.

4 Diskusija

Genetski algoritem je stohastična metoda, zato lahko različni zagoni z enakimi parametri, a drugačnim naključnim semenom, potrebujejo različno število generacij do konvergence. Prednosti pristopa so, da ne potrebuje matematičnega modela ali gradienta, naravno deluje na diskretnih problemih (kot je sestavljanje izraza iz simbolov) in je enostaven za implementacijo; slabosti so, da ne zagotavlja globalnega optimuma in je občutljiv na izbiro parametrov. Elitizem zagotavlja monotonost najboljše najdene rešitve, ima pa znan kompromis: ker se najboljši osebek vedno ohrani nespremenjen, se lahko zmanjša diverziteta populacije in s tem poveča tveganje prezgodnje konvergence v lokalni optimum – pri tej nalogi, kjer je iskalni prostor diskreten in razmeroma majhen, je ta učinek manj izrazit.

5 Zaključek

Implementirani genetski algoritem uspešno najde aritmetični izraz, ki ustreza poljubni dosegljivi ciljni vrednosti, pri čemer zaradi elitizma kriterijska funkcija najboljše rešitve skozi generacije nikoli ne pada. Kadar natančna rešitev ni dosegljiva, program izpiše najboljšo doslej najdeno rešitev, zato je izhod vedno smiseln in notranje konsistenten.

Priloga: Koda

```
1  clc; clear; close all;
2
3  %
4  % =====
5  % VAJA 2 - Genetski algoritem za iskanje aritmetičnega
6  % izraza
7  % =====
8  % Skripta z genetskim algoritmom sestavi aritmetični
9  % izraz iz števk 1-9 in
10 % operacij +, -, *, /, katerega vrednost se čim bolj
11 % približna ciljni
12 % vrednosti. Rezultate izvozi v tabelo LaTeX in graf
13 % poteka kriterijske
14 % funkcije.
15 % =====
16
17 % === PARAMETRI ALGORITMA ===
18 % Priporocene vrednosti: populacija 20-100 osebkov,
19 % 100-1000 generacij,
20 % verjetnost mutacije 0.05-0.2. Vecja populacija pomeni
21 % vecjo zacetno
22 % diverziteto in manjše tveganje prezgodnje konvergence,
23 % a vec racunanja.
24 population_size = 20;      % stevilo osebkov v eni
25 % generaciji (sodo zaradi parjenja)
26 max_generations = 100;    % zgornja meja stevila
27 % generacij
28 mutation_prob = 0.1;      % verjetnost spremembe
29 % posameznega gena
30
31 % Elitizem: najboljši osebek generacije se nespremenjen
32 % prenese naprej.
33 % Posledica je, da najboljša dosežena kriterijska
34 % funkcija nikoli ne pade,
35 % cena pa je manjša diverziteta populacije.
36 use_elitism = true;
37
38 % === UPORABNISKI VNOS ===
39 % Vnos se bere kot niz in pretvori s str2double, ker
40 % privzeti način input()
41 % interno kliče eval() nad vnesenim tekstom, kar v
42 % kombinaciji z lokalnimi
```

```

28 % funkcijami na dnu te skripte v nekaterih okoljih
    povzroci napacen
29 % re-parse datoteke in napako v evaluate_individual.
30 target_value = str2double(input('Vnesi željeno ciljno
    vrednost (npr.:_28):_','s'));
31 while ~(isscalar(target_value) && isfinite(target_value))
32     fprintf('Napačen vnos: ciljna vrednost mora biti š
        tevililo.\n');
33     target_value = str2double(input('Vnesi željeno ciljno
        vrednost (npr.:_28):_','s'));
34 end
35
36 % Najvecje stevilo stevk v izrazu. Pri max_terms = 5
    izraz izgleda npr.
37 % takole: 9 + 5 * 4 - 3 + 2
38 max_terms = str2double(input('Vnesi največje število č
    lenov v izrazu (npr.:_5):_','s'));
39 while ~(isscalar(max_terms) && isfinite(max_terms) &&
    max_terms >= 1 && mod(max_terms,1) == 0)
40     fprintf('Napačen vnos: največje število členov mora
        biti celo število >=_1.\n');
41     max_terms = str2double(input('Vnesi največje število
        členov v izrazu (npr.:_5):_','s'));
42 end
43
44 % === ZACETNA POPULACIJA ===
45 % Prva generacija je naključna, kar zagotovi razprsenost
    po iskalnem prostoru.
46 tic;
47 pop = initialize_population(population_size, max_terms);
48
49 % Zgodovina najboljsega osebka po generacijah za graf in
    tabelo.
50 best_fitness_history = zeros(max_generations,1);
51 best_expressions = strings(max_generations,1);
52 best_values = zeros(max_generations,1);
53
54 % Zastavica loci predcasno najdeno resitev od izteka
    stevila generacij.
55 solution_found = false;
56
57 % === GLAVNA ZANKA GA ===
58 % Ena iteracija zanke je ena generacija.
59 for gen = 1:max_generations
60
61     % Ovrednotenje: vsakemu osebku pripada vrednost
        izraza in kriterijska
62     % funkcija, ki meri, kako blizu cilju je.
63     fitnesses = zeros(population_size,1);

```

```

64     values = zeros(population_size,1);
65
66     for i = 1:population_size
67         [v, f] = evaluate_individual(pop{i}, target_value
68             );
69         values(i) = v;
70         fitnesses(i) = f;
71     end
72
73     % Najboljsi osebek trenutne generacije.
74     [best_fit, idx] = max(fitnesses);
75
76     best_fitness_history(gen) = best_fit;
77     best_expressions(gen) = strjoin(pop{idx}, '');
78     best_values(gen) = values(idx);
79
80     % Ce se vrednost izraza ujema s ciljem, nadaljnje
81     % iskanje nima smisla.
82     if abs(values(idx) - target_value) < 1e-6
83         solution_found = true;
84         break;
85     end
86
87     % === TVORJENJE NOVE GENERACIJE ===
88     % Zaporedje operatorjev GA: selekcija, krizanje,
89     % mutacija.
90     new_pop = cell(population_size,1);
91
92     if use_elitism
93         new_pop{1} = pop{idx};
94         start_i = 2;
95     else
96         start_i = 1;
97     end
98
99     % Otroci nastajajo v parih, ker krizanje iz dveh
100     % starsev tvori dva otroka.
101     for i = start_i:2:population_size
102
103         % Selekcija z ruletnim kolesom: verjetnost izbire
104         % osebk je
105         % sorazmerna z njegovo kriterijsko funkcijo, zato
106         % boljsi osebk
107         % pogostejše prispevajo gene, slabši pa vseeno
108         % ohranijo možnost
109         % izbire in s tem raznolikost genskega zaklada.
110         p1 = roulette_selection(pop, fitnesses);
111         p2 = roulette_selection(pop, fitnesses);

```

```

106         % Krizanje kombinira gradnike dveh delnih resitev
107         [c1, c2] = crossover(p1, p2);
108
109         % Mutacija vnasa nove gene, ki jih v populaciji
110         % ni bilo, in tako
111         % preprecuje zastoje v lokalnem optimumu.
112         new_pop{i} = mutate(c1, mutation_prob);
113
114         if i+1 <= population_size
115             new_pop{i+1} = mutate(c2, mutation_prob);
116         end
117     end
118     pop = new_pop;
119 end
120
121 elapsed_time = toc;
122
123 % === IZPIS REZULTATA ===
124 % Koncni rezultat se izpiše v obeh primerih, tudi če
125 % popolna resitev
126 % ni bila najdena.
127 if solution_found
128     fprintf("\n Rešitev najdena v generaciji %d\n", gen);
129 else
130     fprintf("\n Doseženo največje število generacij (%d)
131     brez popolne rešitve.\n", max_generations);
132     fprintf(" Izpisujem najboljšo doslej najdeno rešitev
133     .\n");
134 end
135 fprintf('Število opravljenih generacij: %d\n', gen);
136 disp(['Izraz: ', char(best_expressions(gen))]);
137 disp(['Vrednost izraza: ', num2str(best_values(gen))]);
138 disp(['Kriterijska funkcija: ', num2str(
139     best_fitness_history(gen))]);
140 fprintf('Čas izvajanja algoritma: %.4f s\n', elapsed_time
141 );
142
143 % === POTI ===
144 % Poti se določi relativno glede na lokacijo te skripte
145 % , zato delovanje
146 % ni odvisno od uporabnikovega računalnika/mape.
147 script_dir = fileparts(mfilename('fullpath'));
148 tab_dir = fullfile(script_dir, 'tabs');
149 fig_dir = fullfile(script_dir, 'figs');
150
151 if ~exist(tab_dir, 'dir')
152     mkdir(tab_dir);

```

```

147 end
148 if ~exist(fig_dir, 'dir')
149     mkdir(fig_dir);
150 end
151
152 % === IZVOZ TABELLE ===
153 tex_file = fullfile(tab_dir, 'najboljsi_rezultati.tex');
154 fid = fopen(tex_file, 'w');
155
156 fprintf(fid, '\\begin{tabular}{|c|l|c|c|}\\hline\\n');
157 fprintf(fid, 'Generacija_&Izraz_&Vrednost_&Kriterijska
    _funkcija_\\\\\\hline\\n');
158
159 for i = 1:gen
160     fprintf(fid, '%d_&s_&%.4f_&%.4f_\\\\\\hline\\n',
        ...
        i, best_expressions(i), best_values(i),
        best_fitness_history(i));
161
162 end
163
164 fprintf(fid, '\\end{tabular}\\n');
165 fclose(fid);
166
167 % === GRAF POTEKA KRITERIJSKE FUNKCIJE ===
168 fig = figure('Color', 'w');
169
170 plot(1:gen, best_fitness_history(1:gen), 'k-', 'LineWidth
    ', 2);
171 xlabel('Generacija', 'FontSize', 12, 'Color', 'k');
172 ylabel('Najboljša_vrednost_kriterijske_funkcije', '
    FontSize', 12, 'Color', 'k');
173 title('Potek_izboljševanja_najboljše_rešitve', 'FontSize'
    , 13, 'Color', 'k');
174 grid on;
175 set(gca, 'FontSize', 12, 'XColor', 'k', 'YColor', 'k', '
    Color', 'w');
176
177 saveas(fig, fullfile(fig_dir, 'potek_kriterijske_funkcije
    .png'));
178
179 % === FUNKCIJE ===
180
181 function pop = initialize_population(N, max_terms)
182     % Razpoložljive aritmetične operacije.
183     ops = {'+', '-', '*', '/'};
184
185     % Populacija je celica, vsak element je en kandidat
        za resitev.
186     pop = cell(N,1);

```



```

187
188     for i = 1:N
189         % Izraz ima obliko stevka operator stevka ...
190             stevka,
191         % torej je skupna dolžina 2*max_terms - 1.
192         expr = cell(1, max_terms*2-1);
193
194         for j = 1:max_terms
195             % Liha mesta so številke 1-9.
196             expr{2*j-1} = num2str(randi([1,9]));
197
198             % Soda mesta so operatorji.
199             if j < max_terms
200                 expr{2*j} = ops{randi(4)};
201             end
202         end
203
204         pop{i} = expr;
205     end
206 end
207
208 function [value, fitness] = evaluate_individual(indiv,
209     target)
210     try
211         % Genotip (celica simbolov) se pretvori v fenotip
212             (niz),
213         % npr. {'9','+', '5','*', '4'} -> '9+5*4'.
214         expr = strjoin(indiv, '');
215
216         value = eval(expr);
217
218         % Kriterijska funkcija 1/(1+|cilj-vrednost|) je
219             padajoca funkcija
220         % absolutne napake, omejena na (0,1]. Popoln
221             zadetek da vrednost 1,
222         % z oddaljenostjo od cilja pa gladko pada proti
223             0, zato je primerna
224         % za ruletno kolo, ki zahteva nenegativne utezi.
225         fitness = 1 / (1 + abs(target - value));
226
227     catch
228         % Neveljaven izraz (npr. deljenje z nič) dobi
229             najslabšo oceno,
230         % s čimer ga selekcija praktično izloči.
231         value = NaN;
232         fitness = 0;
233     end
234 end
235
236
237
238

```

```

229 function selected = roulette_selection(pop, fitnesses)
230     % Ruletno kolo: velikost izseka je sorazmerna
        kriterijski funkciji
231     % osebka, izbira pa je nakljucna, zato je selekcijski
        pritisk mehak -
232     % boljsi osebki so v prednosti, slabši pa niso povsem
        izkljudeni.
233
234     total_fitness = sum(fitnesses);
235
236     % Če so vsi fitnessi 0 (ali je vsota NaN, npr. zaradi
        neveljavnih
237     % izrazov), ruletna izbira ni definirana, zato
        uporabimo enakomerno
238     % nakljucno izbiro med osebki.
239     if total_fitness == 0 || isnan(total_fitness)
240         idx = randi(numel(pop));
241     else
242         probs = fitnesses / total_fitness;      %
            normirane verjetnosti
243         cum_probs = cumsum(probs);              %
            kumulativne verjetnosti
244         r = rand;
245         idx = find(cum_probs >= r, 1, 'first');
246     end
247
248     selected = pop{idx};
249 end
250
251 function [child1, child2] = crossover(p1, p2)
252     % Enotockovno krizanje: izbere se točka reza, repa
        starsev se zamenjata.
253     % Tako se v enem otroku združita gradnika dveh
        različnih delnih resitev.
254
255     % Pri izrazu z eno samo številko (length(p1) == 1)
        krizanje nima smisla,
256     % hkrati bi randi(0) javil napako, zato otroka
        podedujeta starse.
257     if length(p1) <= 1
258         child1 = p1;
259         child2 = p2;
260         return;
261     end
262
263     % Točka reza je na lihem mestu, da ostane zgradba
        izraza smiselna
264     % (številka/operator/številka/...).
265     point = 2*randi(floor(length(p1)/2)) - 1;

```

```

266
267     child1 = [p1(1:point), p2(point+1:end)];
268     child2 = [p2(1:point), p1(point+1:end)];
269 end
270
271 function mutated = mutate(indiv, pm)
272     % Mutacija naključno spremeni posamezen gen. Je edini
      operator, ki v
273     % populacijo vnese vrednost, ki je ni imel noben
      stars, zato vzdrzuje
274     % diverziteto in omogoca izhod iz lokalnega optimuma.
275
276     ops = {'+', '-', '*', '/'};
277
278     for i = 1:length(indiv)
279         if rand < pm
280             if mod(i,2)==0
281                 % Soda mesta so operatorji.
282                 indiv{i} = ops{randi(4)};
283             else
284                 % Liha mesta so stevke.
285                 indiv{i} = num2str(randi(9));
286             end
287         end
288     end
289
290     mutated = indiv;
291 end

```